Software Architecture and Design Specification

Project: Task Manager Application

Version: 1.1

Authors: Team 10 — Sneha Angelin Jisho, Syed Ayaan Hasan, Suraj Acharya, Spoorthi A N, Sumedh B Rao

Date: 09-10-2026

Status: Draft — for review

Revision History

Version	Date	Author	Change Summary
1.0	08-10-2026	Team 10	Initial Software Architecture and Design Specification for the Task Manager Application.
1.1	09-10-2026	Team 10	Consistency audit and cross-check against SRS v1.1 and STP v1.0: added task-assignment, soft-delete/ recovery and role-permission design; aligned 403 handling, hashing (Argon2/bcrypt) and configurable session timeout with the SRS; corrected Figures 3.1 and 4.1-4.3; added Sequence 4 (shared-task update notification); terminology cleanup.

Approvals

Role	Name	Signature/Date
Course Coordinator	Prof. Niveditha	
Team / Project Representative	Syed Ayaan Hasan	

1. Introduction

1.1 Purpose

This document specifies the software architecture and design of the Task Manager Application. It translates the requirements in the SRS into a maintainable browser-based client-server design, defining the major layers, components, data interactions, security architecture, interfaces, key flows, error handling, and user-interface considerations.

1.2 Scope

The system is a responsive web application for creating, organizing, assigning, sharing, and tracking personal and shared tasks. The architecture covers account and authentication, role-based access control, task lifecycle management, labels/categories, search/filtering/sorting, file attachments, collaboration, comments, activity history, dashboard/reporting, notifications, administration, audit logging, and system configuration. Native mobile applications, calendar synchronization, billing, and multi-tenant organization isolation are outside the current scope.

1.3 Audience

Developers, QA engineers, project managers, team members, system administrators, maintenance personnel, course instructors, and assessment evaluators.

1.4 Definitions

SRS — Software Requirements Specification; STP — Software Test Plan; RTM — Requirements Traceability Matrix; UI — User Interface; API — Application Programming Interface; CRUD — Create, Read, Update, Delete; ORM — Object-Relational Mapping; RBAC — Role-Based Access Control; REST — Representational State Transfer; HTTP/HTTPS — Hypertext Transfer Protocol / Secure HTTP; CSRF — Cross-Site Request Forgery; XSS — Cross-Site Scripting; SQL — Structured Query Language; DB — Database; SMTP — Simple Mail Transfer Protocol; TLS — Transport Layer Security; WCAG — Web Content Accessibility Guidelines; ADR — Architecture Decision Record; STRIDE — Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege; UML — Unified Modeling Language; CSV — Comma-Separated Values; UTC — Coordinated Universal Time; JSON — JavaScript Object Notation; JWT — JSON Web Token; Django — Python web framework used by the application. Requirement IDs use the SRS prefixes TM-F (functional), TM-NF (non-functional) and TM-SR (security).

2. Document Overview

2.1 How to use this document

Sections 3 and 4 define the architecture and detailed design. The architectural deliverables are the UML component and sequence diagrams, the architecture decision records (ADRs, Section 3.5), the STRIDE threat model (Section 3.9) and the API design (Section 4.3). The component and sequence diagrams show the principal runtime relationships and flows, including their main failure paths. Error handling, security, UX and traceability describe cross-cutting design decisions. The structure follows ISO/IEC/IEEE 42010 (stakeholders and concerns, Section 3.2); the security architecture draws on OWASP ASVS/Top 10 and NIST SP 800-160 (Section 3.9).

2.2 Related Documents

- Task Manager Application Software Requirements Specification (SRS).
- Task Manager Application Software Test Plan (STP).
- Requirements Traceability Matrix (RTM) contained in the SRS/STP.

3. Architecture

3.1 Goals & Constraints

Architecture goals:

- Clear separation of presentation, application/business logic, and persistence concerns.
- Secure authentication, authorization, task-data protection, auditability, and controlled file handling.
- Maintainability through modular Django components with well-defined responsibilities.
- Reliable notification processing without allowing an external email dependency to destabilize the core application.
- Responsive performance for common task operations (2-second target, TM-NF-001), with a design that can support concurrent users (STP target: 20 simultaneous users, TM-NF-003).
- Accessibility through keyboard navigation, screen-reader labels, high contrast, and clear validation feedback.

Constraints:

- Browser-based access only for this version; no dedicated mobile application.
- Single-organization / single-tenant deployment.
- Django and Django ORM are the application framework/persistence technology specified by the SRS.
- A relational database is required.
- Email service dependency exists for assignment, collaboration, due-date and account (registration, e-mail verification, password-reset) notifications.
- HTTPS is required in production; HTTP may be used during local development.
- No specialized hardware is required.

3.2 Stakeholders & Concerns

Stakeholder	Primary concerns
Team Member	Simple task creation/update, visibility of assigned and shared tasks, comments, notifications, accessibility.
Project Manager	Task assignment, progress monitoring, collaboration permissions, reporting, auditability.
Administrator	User/role administration, audit logs, system configuration, security and system health.
QA Engineers	Requirement traceability, testability, predictable errors, observability and acceptance criteria.
Developers / Maintenance	Modularity, clear interfaces, maintainability, data integrity and diagnosability.
Course / Assessment Stakeholders	Alignment with SRS, traceability, security, reliability and documented design decisions.

3.3 Component (UML) Diagram

The architecture uses a modular layered design (Figure 3.1). The browser communicates with the Django presentation layer; the application layer contains the domain services (authentication, tasks, collaboration, dashboard, administration) and the supporting Audit, Notification and Authorization components; persistence uses Django ORM with a relational database, which also holds the server-side session records. File attachments use dedicated file storage, and notifications are delivered through an external SMTP service, with optional background jobs for scheduled reminders. Dependencies between components are shown as UML «use» relationships.

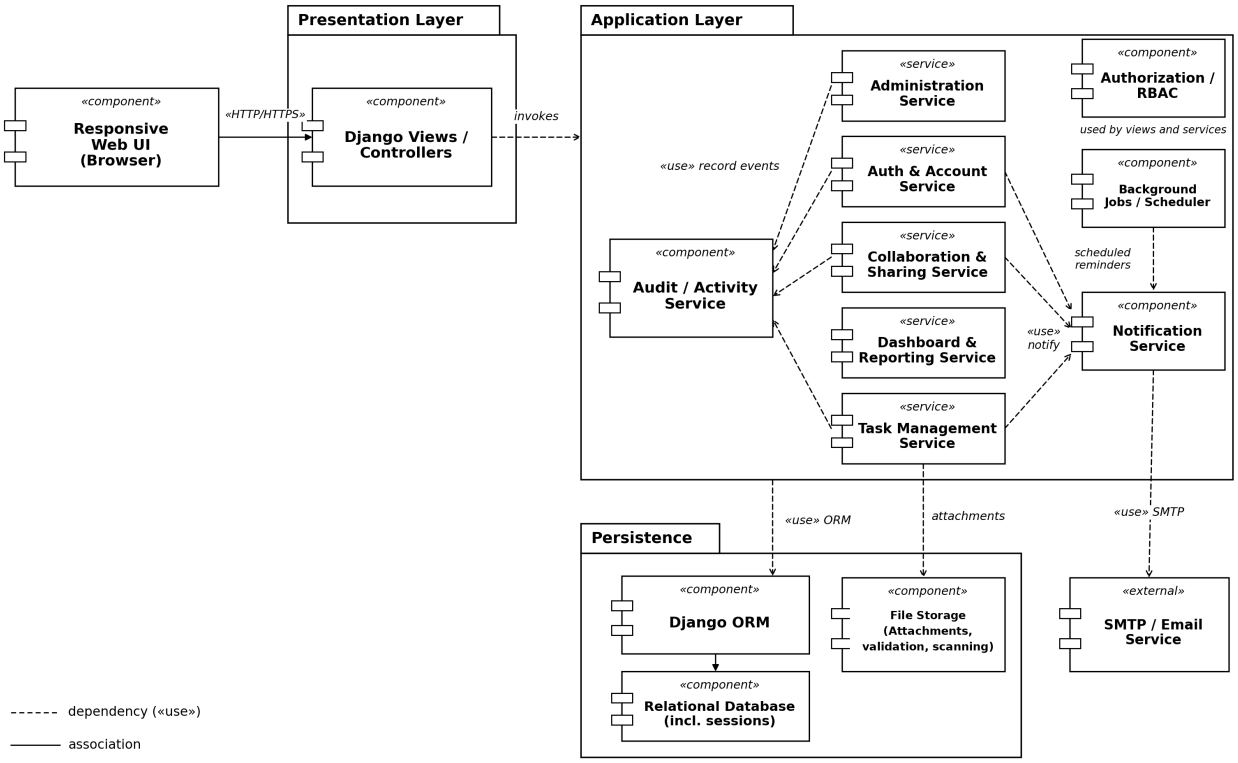


Figure 3.1: Task Manager Application component architecture

3.4 Component Descriptions

Component	Responsibility
Responsive Web UI	Login, dashboard, task lists/details, forms, reports, notifications and administration pages. Provides validation feedback, confirmation messages and role-appropriate navigation.
Django Views / Controllers	Receives browser requests, performs request validation, invokes application services and returns HTML responses or optional JSON responses.
Authorization / RBAC	Role-based and object-level permission checks (role, task owner, Viewer/ Editor share) invoked by views and application services before any protected operation; denies by default and returns 403 on failure (TM-SR-003).
Auth & Account Service	Registration, authentication, profile (name, e-mail with re-verification, avatar)/ password changes, password reset, account lockout and session management using Django's server-side session framework.
Task Management Service	Task creation, editing, deletion (soft delete, recoverable for 30 days), completion/reopening, assignment of tasks to users, due dates, priority, labels/ categories, search/filter/sort and attachment coordination.
Collaboration & Sharing Service	Viewer/Editor sharing, permission changes, access revocation, comments and collaboration events.
Dashboard & Reporting Service	Aggregates open/overdue/completed task information and generates CSV exports of the task list and dashboard summary, and system-wide reports for the Administrator (SRS Fig 7.2).
Administration Service	User administration, global category management, system configuration and administrative controls.
Notification Service	Sends due-date reminders, assignment and collaboration notifications (including updates to shared tasks) and account e-mails (registration confirmation, e-mail re-verification, password reset); in-app delivery is optional. It isolates email delivery from core business operations.
Audit / Activity Service	Records security and task activity events with actor, action and timestamp information; audit records are immutable: no role can edit or delete them (TM-F-052).
Background Jobs / Scheduler	Optional asynchronous processing for scheduled reminders, notification delivery and other jobs that should not block normal web requests.
Django ORM + Relational DB	Persists accounts, roles, tasks, permissions, comments, activity/audit records, configuration and server-side session records (database-backed Django sessions).
File Storage	Stores task attachments after validation and scanning; the SRS specifies a 10 MB maximum attachment size.
SMTP / Email Service	External notification dependency for assignment, collaboration, due-date, registration/verification and password-reset messages.

3.5 Chosen Architecture Pattern and Rationale

A layered, modular monolithic architecture is selected. This directly follows the SRS product perspective, which separates the presentation layer, application layer, and persistence layer. Django provides a natural implementation boundary for these layers while allowing domain responsibilities to be separated into focused modules/services.

A microservices architecture is not selected because the current system is single-tenant, browser-based, and of limited scope. Introducing distributed services would add operational and deployment complexity without a requirement for independent service scaling. Optional background processing is retained for scheduled notifications and other asynchronous work.

Architecture decision records (ADRs):

ADR-001	Server-side Django sessions instead of JWT
Status	Accepted
Context	Sessions must end immediately on logout and after 24 hours of inactivity (TM-SR-004), and accounts lock after repeated failures (TM-F-003). A JWT is stateless and cannot be revoked before it expires without extra infrastructure such as a deny-list.
Decision	Use Django's built-in authentication and server-side session framework (database-backed sessions) with Secure, HttpOnly, SameSite cookies. A session is created at login and deleted at logout; an inactivity timeout enforces the 24-hour expiry (the session expiry is renewed on each request, for example with Django's SESSION_SAVE_EVERY_REQUEST setting. Because SESSION_COOKIE_AGE is a static setting, a small middleware reads the timeout value from system configuration, default 24 hours, and applies it through request.session.set_expiry(), so that TM-F-053 changes apply without a redeploy). The Secure cookie flag is enforced in production and may be relaxed only for local HTTP development.
Alternatives considered	JWT access and refresh tokens with a revocation list (more moving parts, weaker immediate logout); a hybrid of sessions and tokens (unnecessary for this version).
Consequences	Logout and expiry take effect on the next request and no token-revocation infrastructure is needed. Each request performs a session lookup, which is acceptable at the expected scale. If the optional REST endpoints are enabled, they use the same session cookie with CSRF protection; token authentication can be added later for non-browser integrations only.

ADR-002	Layered modular monolith instead of microservices
Status	Accepted
Context	The system is single-tenant, browser-based and of limited scope, built by a small team. The SRS describes presentation, application and persistence layers and has no requirement for independently scaled services.
Decision	Build one Django project organized into modules (Auth, Task, Collaboration, Dashboard, Notification, Administration, Storage; audit recording sits in Administration and RBAC checks in Auth, as in the SRS RTM) in which service-layer classes hold the business rules and views only handle presentation.
Alternatives considered	Microservices with a message broker (rejected for deployment and operational overhead); business logic placed directly in views (rejected for poor testability).
Consequences	Simple deployment and database transactions across modules. Module boundaries are enforced by convention and code review, and the modules can be split into services later if scaling needs change.

ADR-003	Isolated, asynchronous notification delivery
Status	Accepted
Context	E-mail depends on an external SMTP service that can be slow or unavailable (SRS 2.5), while TM-F-014 and TM-F-031 require reminders and collaboration notices within 1 and 5 minutes, and TM-NF-002 requires stable operation.
Decision	Core operations only enqueue a notification event. The Notification Service delivers it through SMTP with timeouts, safe logging and retries, driven by the optional background-job scheduler for reminders.
Alternatives considered	Sending e-mail synchronously inside the request (rejected: an SMTP outage would slow or fail user actions); a third-party notification platform (rejected: out of scope and adds a dependency).
Consequences	User actions stay fast and succeed regardless of e-mail availability. Delivery status must be tracked and monitored, and the background-job mechanism must be chosen (open issue in 4.6).

3.6 Technology Stack & Data Stores

Area	Technology / Design
Client	HTML/CSS/JavaScript delivered through Django templates; responsive browser UI.
Application framework	Python with Django.
Persistence	Django ORM with a relational database.
Authentication/session	Django authentication/session mechanisms; secure server-side
Email	SMTP/email service for notifications and password-reset messages.
File storage	Application-managed file storage for task attachments, with validation/ scanning before storage.
Optional integration	REST-style authenticated JSON endpoints if enabled for future integrations.
Logging/monitoring	Application logging plus audit/activity records and operational monitoring components.

3.7 Risks & Mitigations

Risk	Mitigation
Email service unavailable	Use timeout/error handling and retry behavior; keep core task operations independent of immediate email delivery.
Unauthorized task access	Enforce RBAC and object-level authorization in the application layer; test cross-user/cross-role access.
Malicious file/input	Validate and sanitize input; scan attachments before storage; enforce the 10 MB attachment limit.
Database failure or corruption	Use transactions for critical updates, backups appropriate to deployment, and data-integrity checks.
Performance degradation	Keep common operations within the response-time target in the SRS (TM-NF-001), optimize queries, and use load testing for concurrency.
Audit-log tampering	Restrict audit writes to controlled application paths and make audit entries immutable for every role.
Session compromise	Use HTTPS in production, secure server-side sessions, logout invalidation and 24-hour inactivity expiry.

3.8 Traceability to Requirements

Requirement(s)	Area	Primary design element	Design responsibility
TM-F-001–005	Authentication & Account	Auth & Account Service	Registration (with confirmation e-mail), login, lockout, password reset (30-minute single-use link), profile (name, e-mail with re-verification, avatar) and password change.
TM-F-010–014	Task Lifecycle	Task Management Service / Notification Service	Create, edit, delete (30-day recovery), complete/reopen, assign, reminders.
TM-F-020–022	Organization & Attachments	Task Management Service / File Storage	Labels, search/filter/sort, attachments.
TM-F-030–034	Collaboration	Collaboration & Sharing Service / Notification / Audit	Sharing, notifications, revocation, comments, activity history. Per the SRS RTM, TM-F-034 is owned by TaskModule and TM-F-031 by NotifModule.
TM-F-040–041	Dashboard & Reporting	Dashboard & Reporting Service	Dashboard summaries and CSV export.
TM-F-050–053	Administration	Administration Service / Audit	User administration, categories, audit log, system configuration.
TM-NF-001–005	Quality Attributes	Web UI / Application / Persistence / Infra	Performance, reliability, concurrency, accessibility, retention.
TM-SR-001–005	Security	Security controls across all layers	TLS, password hashing, RBAC, session expiry, input validation.

3.9 Security Architecture

Security is treated as a cross-cutting concern rather than a single module.

Threat model (STRIDE):

STRIDE category	Threat in this system	Mitigation (SRS reference)
Spoofing	An attacker impersonates a user with guessed or stolen credentials, or a hijacked session.	Salted adaptive password hashing with Argon2/bcrypt (TM-SR-002); 5-attempt / 15-minute lockout (TM-F-003); server-side sessions invalidated on logout and after 24 h inactivity (TM-SR-004); Secure/HttpOnly cookies; time-limited (30 min), single-use reset links (TM-F-004).
Tampering	Task data, permissions or in-transit requests are modified, including forged cross-site requests.	TLS 1.2+ (TM-SR-001); CSRF tokens on all state-changing requests; RBAC and object-level checks (TM-SR-003); parameterized ORM queries and input validation (TM-SR-005); transactions for multi-step updates.
Repudiation	A user or administrator denies an action such as a deletion or permission change.	Audit / Activity Service records actor, action, target and timestamp (TM-F-034, TM-F-052); audit entries are immutable for every role (TM-F-052); administrative changes, including system-configuration changes, are logged (TM-F-050, TM-F-053).
Information disclosure	Credentials, tasks or attachments reach unauthorized users through access gaps, logs or error messages.	RBAC and share-permission checks; TLS in transit; no passwords or session secrets in logs; generic authentication errors (TM-F-002); attachments served only through authorized views; at-rest protection defined by deployment configuration (open issue 4.6).
Denial of service	Brute-force logins, upload flooding, request floods, or an e-mail outage degrade the system.	Account lockout; 10 MB attachment limit (TM-F-022); paginated task lists; notification delivery isolated behind timeouts, retries and background jobs (ADR-003); rate limiting at the reverse proxy / web server (deployment).
Elevation of privilege	A Team Member acts as a Project Manager or Administrator, or reaches another user's tasks by calling URLs or endpoints directly.	RBAC enforced server-side on every view and endpoint, not only in the UI (TM-SR-003); object-level checks for owner and Viewer/Editor; administration restricted to the Administrator role; role changes audited; uploads validated and scanned before storage.

Security controls:

- Transport security: HTTPS with TLS 1.2+ in production; local development may use HTTP as permitted by the SRS.
- Credential protection: passwords are hashed with Argon2 (or bcrypt) through Django's PASSWORD_HASHERS setting, because Django's default hasher is PBKDF2 (TM-SR-002); plaintext passwords are never stored.
- Authentication/session security: server-side sessions, immediate invalidation on logout, and automatic expiry after 24 hours of inactivity.

- Authorization: RBAC plus task-level permission checks ensure users act only on tasks they own or have been granted access to.
- Input security: validate and sanitize user-controlled values to mitigate SQL injection, XSS and similar attacks; rely on parameterized ORM operations.
- File security: validate attachment size/type as appropriate and scan files before storage.
- Auditability: record security and administrative events with actor, action and timestamp; no role can edit or delete audit history through the application.
- Data protection: protect account and task data from unauthorized access in transit and at rest. The SRS does not mandate a specific at-rest encryption mechanism, so deployment configuration should define it.
- Logging: do not place passwords, session secrets or other sensitive credentials in application logs.

4. Design

4.1 Design Overview

The design organizes the Django application into modules corresponding to the functional areas identified in the SRS and Test Plan. Views handle presentation concerns, application services implement business rules and authorization, and persistence uses Django ORM. Cross-cutting services provide authorization, notifications, auditing and validation. This structure keeps user-interface code separate from domain logic and makes individual modules easier to unit and integration test.

Module	Key responsibilities	Main data
AuthModule	Registration, authentication (disabled accounts rejected), logout, lockout, reset, profile, password changes, sessions, and RBAC / object-level permission checks (TM-SR-003).	User, Role, Session, PasswordResetToken
TaskModule	Task CRUD (soft delete, 30-day recovery), assignment to users, status, due date, priority, labels/categories, search/filter/sort, per-task activity history (status changes, edits, comments, assignments).	Task (assignee, deleted_at), Label/Category, TaskStatus, activity history
CollabModule	Sharing, Viewer/Editor permissions, revoke, comments.	TaskShare, Comment
DashModule	Dashboard aggregates, CSV reporting and system-wide reports for the Administrator.	Derived task metrics / report rows
NotifModule	Due reminders, assignment and collaboration notifications, account e-mails.	Notification / delivery status
AdminModule	User management (view, disable, delete), global category management (create, rename, archive), system configuration applied without redeploy, searchable audit log of security events, audit recording and retention.	User, Category, SystemConfig, AuditEvent
StorageModule	Attachment validation, scanning and storage/ retrieval.	Attachment metadata + file object

Labels and categories (TM-F-020, TM-F-051): users assign labels/categories to tasks; the global category/label list is managed by the Administrator (SRS Fig 7.2), and archived categories no longer appear when a task is created.

Role permissions (SRS 2.3; the Guest role follows TM-F-001, TM-F-002 and TM-F-004):

- Guest (not logged in): can register an account (TM-F-001), log in (TM-F-002) and reset a forgotten password (TM-F-004); no other functions are available.
- Team Member: creates and updates own and assigned tasks, comments on tasks they can access, and acts on shared tasks according to the Viewer or Editor permission.
- Project Manager: assigns tasks, monitors task progress, reviews reports, and manages task access and collaboration.

• Administrator: manages users and roles, reviews the audit log, configures system settings and monitors system health. The SRS use-case diagram (Fig 7.2) also shows the Administrator viewing system reports, configuring categories/labels and managing collaborator permissions.

4.2 UML Sequence Diagrams

The following sequences represent core flows that are directly supported by the SRS. Each diagram uses UML combined fragments (alt / opt) to show the main alternative paths, and the numbered messages follow the order of execution.

Sequence 1 — User Authentication

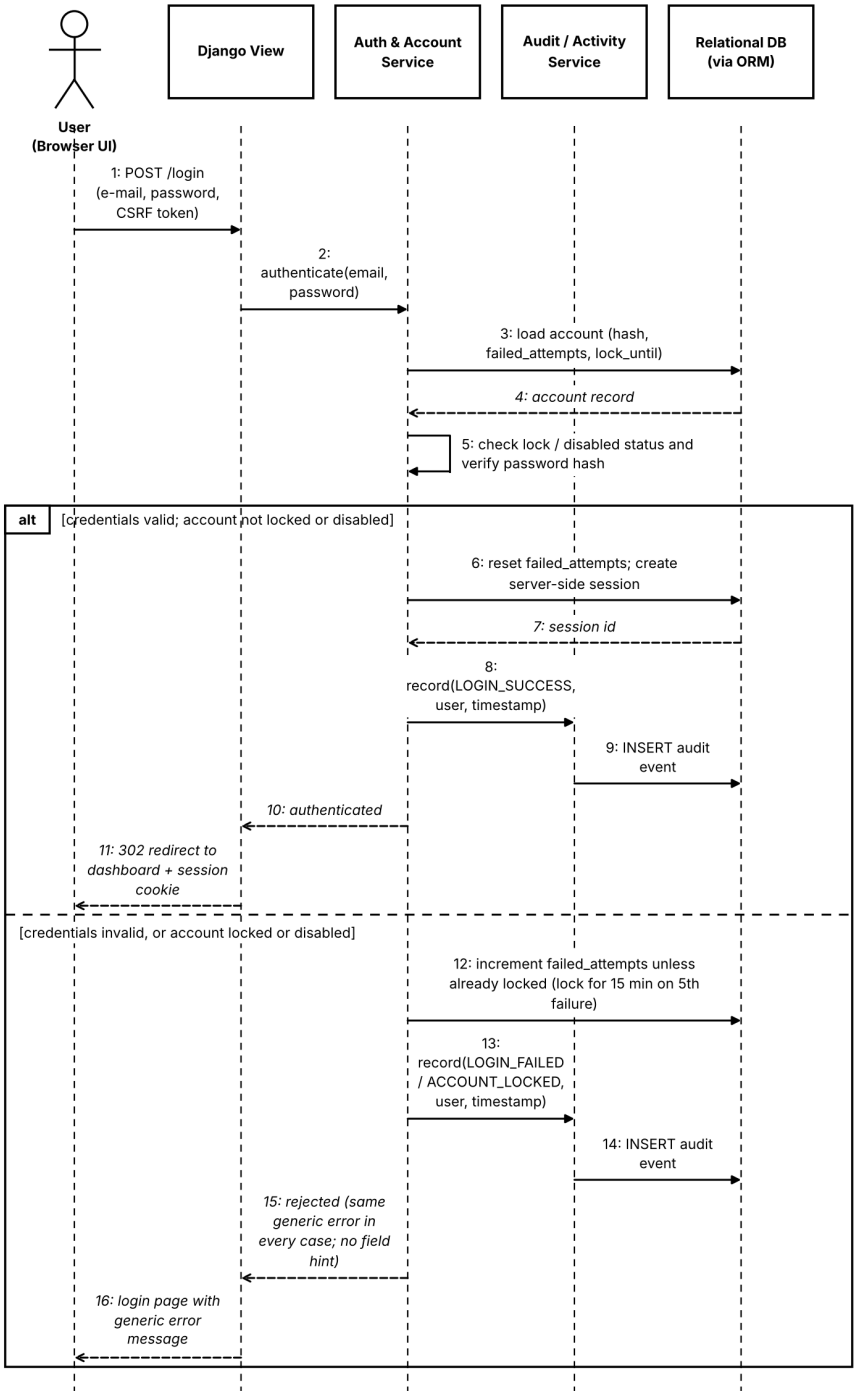


Figure 4.1: Authentication and session creation sequence, including the failed-login and lockout path

Sequence 2 — Create Task

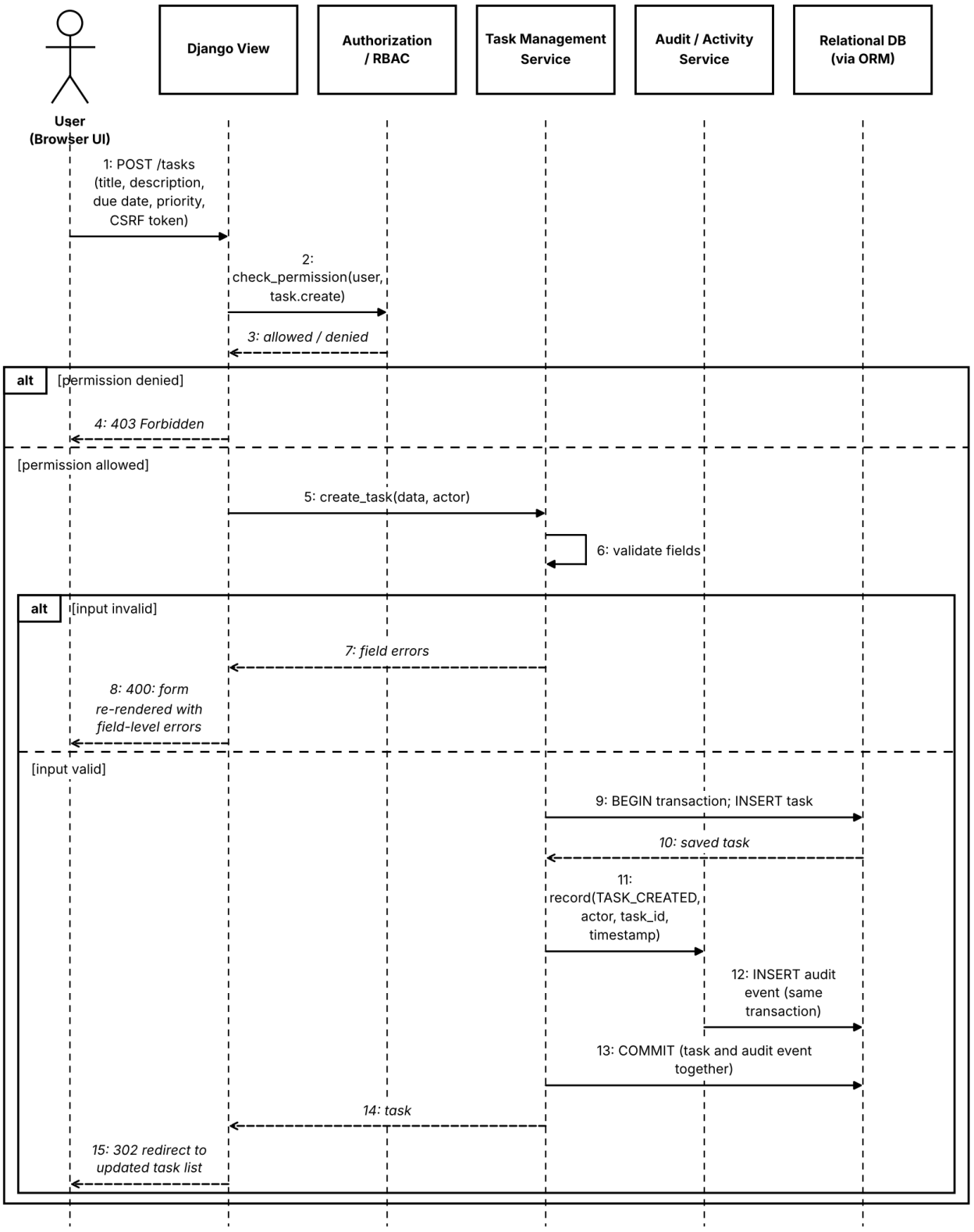


Figure 4.2: Task creation and audit sequence, including permission-denied and validation-failure paths

Sequence 3 — Share Task and Notify Collaborator

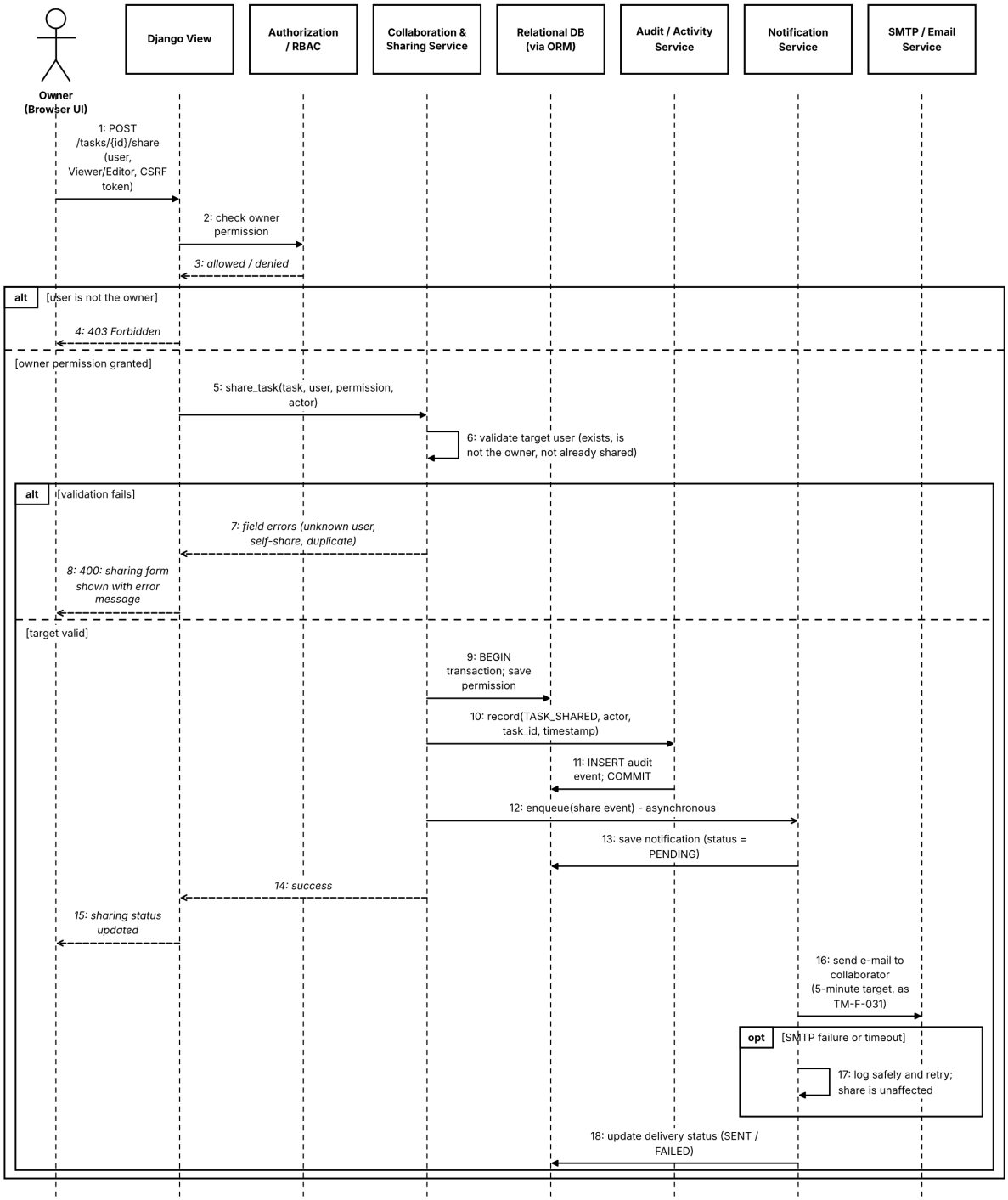


Figure 4.3: Task sharing and collaborator notification sequence, including notification-failure handling

Sequence 4 — Update Shared Task and Notify Collaborators

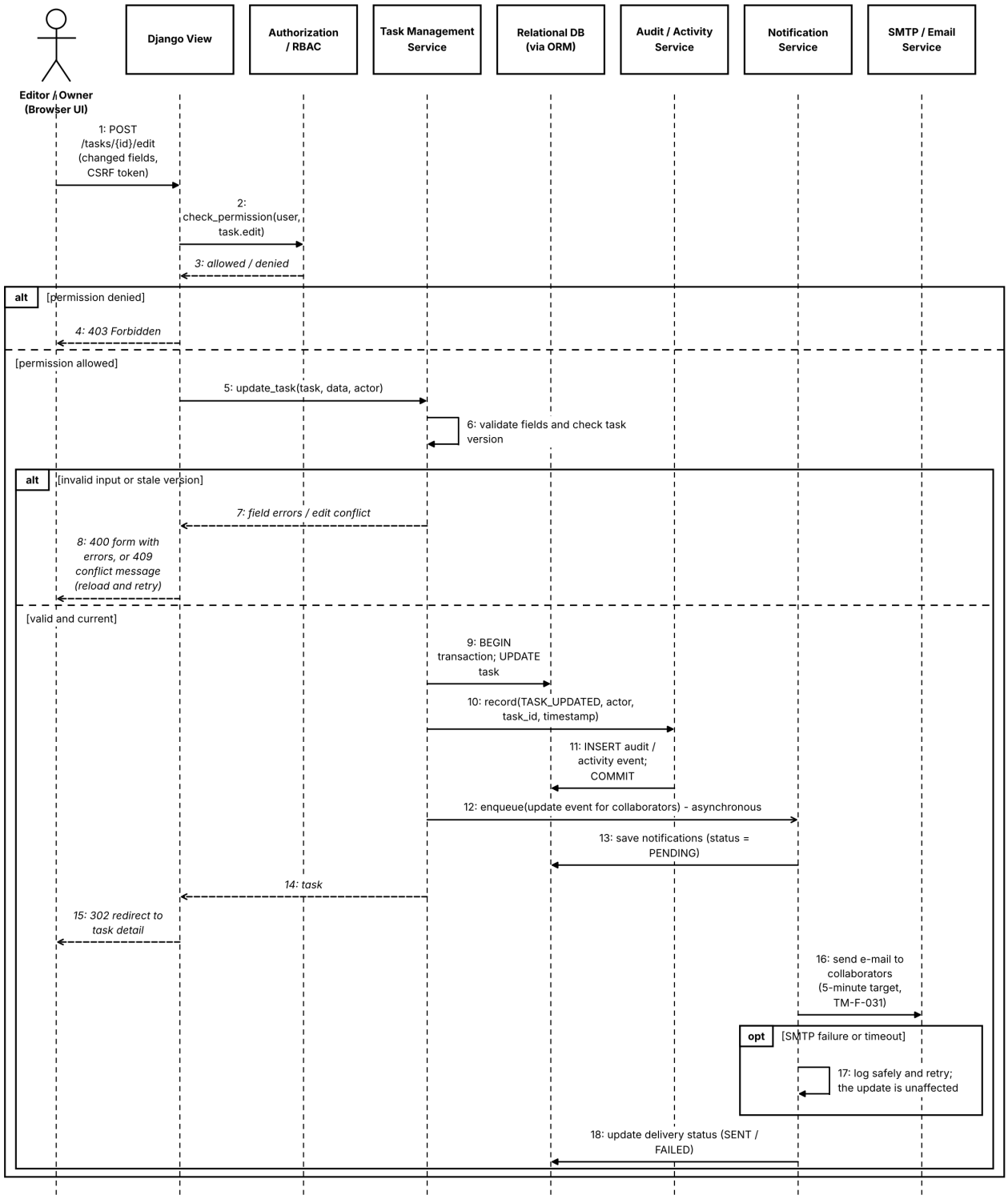


Figure 4.4: Shared-task update and collaborator notification sequence (TM-F-031), including conflict and notification-failure handling

4.3 API Design

The SRS lists REST API endpoints as optional/future integration interfaces. The following is therefore an interface design for use if the REST layer is enabled; the core application does not depend on exposing every service as a public API. The examples are not exhaustive (logout, task listing, deletion, comments and attachments follow the

same conventions). Endpoints use the same server-side session cookie as the web UI and require the CSRF token on state-changing requests.

Method	Endpoint	Request	Success response	Typical errors
POST	/api/auth/login	{email, password}	{status, sessionEstablished}	401 invalid credentials or locked account (same generic message)
POST	/api/tasks	{title, description, dueDate, priority, assigneeId (optional)}	{id, status, task}	400 validation; 401/403 authorization
PATCH	/api/tasks/{id}	Partial task fields (including assigneeId)	{id, status, task}	400 validation; 403 forbidden; 404 task not found; 409 edit conflict
POST	/api/tasks/{id}/share	{userId, permission}	{status, share}	400 validation (unknown user, self-share, duplicate); 403 not the owner; 404 task not found
GET	/api/dashboard?from=&to=	Query range	{open, overdue, completed}	400 invalid range; 401 unauthenticated

Internal component interfaces:

- TaskService.create/update/delete/complete/assign(task, actor): validates business rules, checks authorization and persists changes. Assignment is restricted to the Project Manager role; updates to a shared task and new assignments enqueue notifications to the affected users (TM-F-031).
- NotificationService.enqueue/send(event): accepts a domain notification event and delivers it asynchronously where configured.
- AuditService.record(actor, action, target, timestamp): creates an immutable audit/activity record; timestamps are stored in UTC.

4.4 Error Handling, Logging & Monitoring

Error handling:

- Validate input at the boundary and return clear field-level validation messages.
- Use consistent authorization failures: unauthorized actions return 403 (TM-SR-003) and 404 is used only for resources that do not exist; error responses never include task details. Login failures use one generic message whatever the cause (TM-F-002).
- Use standard HTTP semantics for optional REST endpoints: 400 for invalid requests, 401 for unauthenticated requests, 403 for unauthorized actions, 404 for missing resources, 409 for edit conflicts (stale updates), and 500 for unexpected server failures.
- Wrap critical multi-step database updates in transactions so partial updates do not leave inconsistent task or permission state. Task creation, sharing and similar operations write their audit event in the same transaction.
- External notification failures must time out, be logged safely, and follow the configured retry behavior without crashing the main request path.
- Concurrent edits (TM-NF-003): tasks carry a version or last-updated value; an update based on a stale version is rejected with 409 (or a form message in the web UI) and the user is asked to reload before retrying.

Logging and monitoring:

- Application logs capture operational errors and significant events without passwords, session secrets or other sensitive credentials.
- Audit logs capture security/administrative actions and task activity required by the SRS.
- Monitor response time, error rate, authentication failures/lockouts, notification failures, job failures and database/storage health.
- Deleting a task is a soft delete: it disappears from all views and can be recovered for 30 days (TM-F-012). Audit logs and deleted-task records are retained for at least the SRS retention period of 30 days (TM-NF-005).

4.5 UX Design

- Responsive layouts for desktop, tablet and mobile browser viewports.
- Role-aware navigation: Team Members, Project Managers and Administrators see only relevant functions.

- Task list supports clear status, priority, due date, labels and sorting/filtering controls.
- Forms provide clear labels, validation feedback, confirmation messages and safe defaults.
- Task deletion requires confirmation.
- Accessibility follows the SRS WCAG 2.1 AA target: keyboard navigation, screen-reader labels and high-contrast presentation.
- Dashboard presents open, overdue and completed task summaries over a chosen date range.
- Notifications clearly distinguish due-date reminders and collaboration updates.

4.6 Open Issues & Next Steps

- Confirm the final production relational database choice; the SRS specifies a relational database but does not mandate a specific vendor.
- Confirm the background-job implementation (if used) for scheduled reminders and notification retries.
- Confirm the production file-storage backend and malware-scanning mechanism.
- Define the supported browser matrix for final compatibility testing.
- Finalize whether and when optional REST endpoints will be enabled.
- Define deployment-specific monitoring, backup and at-rest encryption controls consistent with the security objectives.
- Define rate limiting for the login and file-upload endpoints at the reverse proxy / web-server level (referenced in the STRIDE denial-of-service mitigation).
- Confirm whether the "projects" mentioned in SRS 2.3 and 3.1 need their own entity; no SRS requirement defines one.
- Confirm who may recover a soft-deleted task during the 30-day window (owner or Administrator) and whether a Project Manager may assign a task to any user.
- Confirm which roles may change collaborator permissions: SRS Fig 7.2 shows the Administrator doing so, while TM-F-030 and TM-F-032 give sharing and revocation to the task owner and SRS 2.3 to the Project Manager.

5. Appendices

5.1 Glossary

All abbreviations and terms used in this document, including Django, ORM, RBAC, SMTP, TLS, REST, RTM, SRS, STP, ADR, STRIDE and UML, are defined in Section 1.4.

5.2 References

- Task Manager Application SRS v1.1, Team 10, dated 06-09-2026.
- Task Manager Application Software Test Plan v1.0, Team 10, dated 28-09-2026.
- ISO/IEC/IEEE 42010 — Systems and software engineering: Architecture description.
- OWASP Application Security Verification Standard (ASVS) and OWASP Top 10.
- NIST SP 800-160 — Systems Security Engineering.
- Django documentation — authentication, sessions and security (docs.djangoproject.com).

5.3 Tools

Development/design tools consistent with the project documents may include Django development tooling, a relational database, diagramming tools such as PlantUML/draw.io, browser developer tools, API testing tools where REST endpoints are enabled, and the testing tools identified in the STP (pytest/Django test runner, Selenium or Playwright, Postman, Locust/JMeter, axe/Lighthouse, and OWASP ZAP or Burp Suite).